

DEVOPS MASTER COURSE

Volume 7E

Azure Enterprise DevOps, Security & Governance

Chapters 41-55

Infrastructure as Code • Continuous Delivery • Observability • Security Operations • Landing Zones • Resilience • FinOps

2026 Edition

About This Part

Volume 7E develops the enterprise control plane around Azure workloads. Earlier parts established identity, infrastructure, data platforms, containers, serverless compute, and integration. This part turns those capabilities into an operable cloud platform through declarative delivery, secure pipelines, telemetry, policy, threat detection, incident response, landing-zone governance, resilience engineering, and cost accountability.

The goal is not to memorize product blades. It is to build a repeatable operating model in which every change is reviewed, every deployment is traceable, every workload is observable, every privilege is bounded, every control produces evidence, and every recovery procedure is tested.

Safety, Security, and Cost Controls

- Use sandbox subscriptions, budgets, owner tags, expiry dates, and approved regions before provisioning labs.
- Use workload identity federation, managed identities, and short-lived credentials. Do not store client secrets, publish profiles, PATs, or access keys in repositories or pipeline variables.
- Treat policy exemptions, break-glass access, security suppressions, firewall changes, and release overrides as time-bound, owned, and auditable.
- Filter sensitive telemetry, minimize retention, and avoid placing credentials, tokens, personal data, or regulated payloads in logs and traces.
- Use synthetic data and controlled test systems. Do not run destructive deployment-stack, policy-remediation, failover, or incident-response exercises in production.
- Validate current service availability, licensing, pricing, limits, preview status, and organizational standards before production use.

Table of Contents

Chapter 41	Azure Resource Manager, Bicep, and Deployment Stacks
Chapter 42	Azure Platform Engineering, Template Specs, and Deployment Environments
Chapter 43	Azure DevOps Organizations, Repos, Boards, and Artifacts
Chapter 44	Azure Pipelines Continuous Integration and Agent Security
Chapter 45	Azure Pipelines Continuous Delivery, Environments, and Release Strategies
Chapter 46	Azure Monitor, Log Analytics, Metrics, Alerts, and KQL
Chapter 47	Application Insights, OpenTelemetry, SLOs, and Distributed Tracing
Chapter 48	Azure Audit, Resource Graph, Automation, and Fleet Operations
Chapter 49	Azure Key Vault, Managed HSM, Secrets, Keys, and Certificates
Chapter 50	Microsoft Defender for Cloud and Cloud Security Posture Management
Chapter 51	Microsoft Sentinel and Security Operations Engineering
Chapter 52	Azure Network Protection and Zero Trust Enforcement
Chapter 53	Azure Landing Zones, Policy, and Governance at Scale
Chapter 54	Azure Well-Architected, Resilience Engineering, and FinOps
Chapter 55	Enterprise Azure Platform and Secure Delivery Capstone

Navigation: Heading styles support the document outline in Microsoft Word and LibreOffice. The final appendix provides command, readiness, troubleshooting, glossary, and official documentation references.

Chapter 41 - Azure Resource Manager, Bicep, and Deployment Stacks

Designing deterministic Azure infrastructure lifecycle with modular Bicep, safe change review, deployment stacks, and policy-aware automation

Learning Objectives

- Explain ARM control-plane behavior, scopes, idempotent declarations, dependency resolution, and deployment history.
- Build reusable Bicep modules with typed parameters, outputs, validation, and secure defaults.
- Use what-if, deployment stacks, locks, policy, and staged rollouts to reduce destructive change risk.
- Troubleshoot authorization, provider registration, quota, policy, dependency, and lifecycle errors.

41.1 Declarative Infrastructure and the ARM Control Plane

Azure Resource Manager is the control plane through which Azure resources are created, updated, tagged, locked, and deleted. Bicep is a domain-specific language that compiles to ARM JSON while providing a cleaner authoring experience. A production design separates desired state, environment parameters, policy, deployment identity, and operational evidence.

- Keep resource names, locations, SKU choices, network exposure, identities, diagnostic settings, tags, and locks explicit.
- Deploy at the narrowest appropriate scope: resource group, subscription, management group, or tenant.
- Treat outputs as contracts between modules, not as a place to expose secrets.
- Use modules for stable capabilities such as networking, monitoring, identity, and application landing zones.

41.2 Safe Change and Resource Lifecycle

Run linting, validation, policy checks, and what-if before deployment. Review what Azure will add, modify, ignore, or delete. Incremental deployments do not automatically delete resources removed from a template. Deployment stacks add managed-resource lifecycle behavior and can detach or delete resources according to explicit settings.

Control	Purpose	Production Rule
what-if	Preview intended changes	Require review for identity, network, data, policy, and deletion changes.
Deployment stack	Manage a declared resource collection	Use explicit deny settings and delete/detach behavior; test in lower environments.
Resource lock	Protect critical resources	Apply selectively; document operational procedures and emergency removal.
Azure Policy	Deny, audit, modify, or deploy controls	Test effects and remediation identity before broad assignment.

```
az bicep build --file main.bicep
az deployment sub what-if --location eastus --template-file main.bicep --parameters @prod.bicepparam
az stack sub create --name platform-prod --location eastus --template-file main.bicep --parameters @prod.bicepparam --
deny-settings-mode none --action-on-unmanage detachAll
```

41.3 Module Quality and Release Evidence

A module should have a clear owner, semantic version, compatibility notes, tests, examples, policy expectations, and upgrade path. Pin module versions and provider API versions intentionally. Record source revision, module versions, parameter set, deployment identity, what-if result, approvals, deployment operation IDs, and post-deployment validation.

- Never pass secrets in plain parameter files or command history. Reference Key Vault or use runtime identity.
- Avoid implicit cross-resource assumptions. Model dependencies and output contracts explicitly.
- Use a controlled module registry or template spec and scan source before publication.
- Test rollback as a forward corrective deployment; ARM does not provide a universal transactional rollback.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
What-if shows unexpected deletion	Wrong scope, stack lifecycle option, renamed resource, loop/key change, or conditional false	Stop. Compare resource IDs and prior state, review stack managed resources, fix identity keys, and rerun what-if.
Deployment denied	Azure Policy, RBAC, lock, deny assignment, provider registration, or quota	Inspect deployment operations and policy details; verify deployment identity and scope; request controlled remediation rather than bypassing controls.
Module works in dev but fails in prod	Different policy, region, feature registration, naming, quota, or parameter assumptions	Compare effective policy, allowed locations/SKUs, provider state, quotas, API versions, and parameter evidence.
Resource remains after removal from Bicep	Incremental deployment semantics or stack detach behavior	Decide whether ownership belongs to the stack; use an approved deletion workflow after dependency and data-retention review.

Hands-on Lab 41: Deploy a Governed Bicep Stack with What-If

Prerequisites

- Sandbox subscription and Azure CLI with Bicep.
- Permission to create a resource group, storage account, Log Analytics workspace, and deployment stack.

Implementation

1. Create modular Bicep for a tagged resource group workload with diagnostics and secure network defaults.
2. Create development and production parameter files without secrets.
3. Run lint, build, validate, and what-if; save output as release evidence.
4. Deploy to development and verify resource IDs, tags, diagnostics, and policy compliance.
5. Create a deployment stack and inspect managed resources.
6. Remove one noncritical resource from the declaration and compare detach versus delete behavior in a disposable environment.
7. Document corrective-deployment and emergency lock procedures.

Validation Evidence

- What-if matches the reviewed change.
- Stack-managed resources and lifecycle settings are recorded.
- Policy and RBAC failures are visible and explainable.
- No secrets appear in source, parameters, logs, or shell history.

Cleanup

- Delete the disposable stack and resources using the approved lifecycle option.
- Remove temporary assignments, locks, and generated parameter files.

Interview Questions and Model Answers

1. Why use Bicep instead of raw ARM JSON?

Bicep provides concise syntax, type safety, modules, tooling, and the same ARM capabilities because it compiles to ARM JSON.

2. What problem do deployment stacks solve?

They track a managed set of resources and support explicit lifecycle and deny settings beyond ordinary deployment history.

3. Why is what-if not sufficient by itself?

It is a critical preview, but production safety also requires policy evaluation, ownership, data-impact review, approvals, tests, and post-deployment validation.

4. How do you roll back an infrastructure deployment?

Prefer a reviewed forward fix or redeployment of a known-good declaration; separately protect and restore stateful data when required.

Chapter Summary

- Use Bicep modules and narrow scopes to create deterministic declarations.
- Review every change with what-if and policy-aware gates.
- Use deployment stacks carefully for resource lifecycle, not as an excuse for broad deletion.
- Preserve complete release and validation evidence.

Chapter 42 - Azure Platform Engineering, Template Specs, and Deployment Environments

Creating paved roads, reusable environment definitions, catalog governance, self-service boundaries, and platform-product operating models

Learning Objectives

- Distinguish a cloud platform product from a collection of shared scripts.
- Publish versioned infrastructure capabilities through template specs, registries, or approved catalogs.
- Design self-service environments with policy, identity, budget, expiry, and ownership boundaries.
- Define platform SLOs, adoption metrics, support models, and deprecation processes.

42.1 Platform as a Product

A platform team reduces cognitive load by providing secure defaults, automated paths, documentation, support, and measurable service levels. It must not become an unresponsive ticket queue or hide critical constraints. The platform contract covers what developers can request, what the platform guarantees, and what the workload team still owns.

Platform Capability	Platform Owns	Workload Team Owns
Subscription vending	Hierarchy, policy, logging, connectivity, budget baseline	Application data classification and requested exceptions.
Application landing zone	Network, identities, diagnostics, secret integration	Application configuration, scaling, SLO, release, and runbooks.
CI/CD templates	Secure tasks, identities, evidence, gates	Tests, artifact content, deployment choice, and business validation.
Observability starter	Workspace routing, dashboards, alerts-as-code	Meaningful telemetry, service ownership, and on-call response.

42.2 Versioned Catalog and Environment Lifecycle

Template Specs and private Bicep registries can publish approved infrastructure building blocks. Azure Deployment Environments can expose curated environment definitions to development teams while retaining centralized governance. Whatever product is used, the organization needs versioning, compatibility, promotion, rollback, and retirement rules.

- Use immutable versions and aliases only as convenience pointers. Production records must identify the exact version.
- Require owner, data classification, cost center, environment, region, and expiry metadata during provisioning.
- Run policy, security, cost, quota, and architecture checks before environment creation.
- Automate suspension or deletion only after notification, data-retention checks, and exception handling.

42.3 Measuring Platform Value

Measure lead time to first compliant environment, deployment success, golden-path adoption, exception volume, support demand, reliability, cost per tenant, and developer satisfaction. Avoid vanity metrics such as number of templates. A good platform makes the safe path faster than a custom path.

1. Publish a small supported product with explicit service boundaries.
2. Onboard pilot teams and observe where the path fails.
3. Automate repeated support work and improve documentation.
4. Measure reliability and developer outcomes.
5. Deprecate unsafe or unused variants with migration support.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Self-service request is blocked	Policy, quota, catalog version, permission, or missing metadata	Surface the exact gate and owner; provide compliant alternatives instead of generic failure.
Teams bypass the platform	Golden path is slow, inflexible, undocumented, or missing required capability	Interview users, compare lead time and failure rate, remove unnecessary friction, and publish extension points.
Environment cleanup causes data loss	Expiry automation ignored classification, retention, backup, or dependency	Pause automation, restore if possible, add approval and data-impact checks, and test the lifecycle in disposable subscriptions.
Template update breaks consumers	Unversioned change or undocumented contract	Restore known-good version, publish a compatibility matrix, and introduce semantic versioning and migration tests.

Hands-on Lab 42: Publish a Versioned Application Landing-Zone Product

Prerequisites

- A sandbox subscription and source repository.
- Permission to create a template spec or publish a Bicep module.

Implementation

1. Define the product contract: inputs, outputs, defaults, policy, supported regions, ownership, and SLO.
2. Build a Bicep module for a resource group, managed identity, Key Vault access, monitoring, tags, and budget-ready metadata.
3. Publish version 1.0.0 to an approved internal distribution mechanism.
4. Create a consumer deployment that pins the exact version.
5. Introduce a compatible version and prove the old consumer still deploys.
6. Add an intentionally incompatible change in a separate major version and document migration.
7. Record adoption, support, and deprecation metrics.

Validation Evidence

- Consumers deploy without copying module source.
- Version pinning makes releases reproducible.
- Policy and ownership metadata are enforced.
- Migration and deprecation paths are documented.

Cleanup

- Remove test environments and unpublished versions.
- Retain release notes and validation evidence.

Interview Questions and Model Answers

1. What is a paved road?

A supported, automated path with secure defaults, documentation, observability, and clear ownership that teams can use with low friction.

2. How do template specs and private Bicep registries differ?

Both distribute reusable declarations; template specs are ARM resources for versioned templates, while a Bicep registry distributes Bicep modules as OCI artifacts.

3. What should a platform team measure?

Developer lead time, successful adoption, platform reliability, exception demand, support load, cost, and user satisfaction.

4. How do you prevent self-service from becoming uncontrolled provisioning?

Enforce identity, policy, budget, metadata, catalog, network, security, quota, expiry, and audit controls before and after provisioning.

Chapter Summary

- Treat the platform as a product with consumers and SLOs.
- Publish immutable, versioned infrastructure capabilities.
- Bound self-service with policy, identity, ownership, cost, and lifecycle controls.
- Measure developer and operational outcomes, not template count.

Chapter 43 - Azure DevOps Organizations, Repos, Boards, and Artifacts

Designing traceable engineering work, repository governance, package provenance, access boundaries, and lifecycle controls

Learning Objectives

- Structure Azure DevOps organizations and projects around ownership and isolation requirements.
- Configure repository protections, branch policies, identity scopes, and secret prevention.
- Connect work items, commits, builds, packages, releases, incidents, and evidence.
- Operate Azure Artifacts feeds with provenance, retention, upstream-source, and permission controls.

43.1 Organization and Project Boundaries

An Azure DevOps organization is a major administrative and identity boundary. Projects organize repositories, pipelines, boards, feeds, teams, and permissions. Avoid one project per microservice without reason, but also avoid a single project that allows unrelated teams to inherit broad access.

- Use Microsoft Entra groups for membership and lifecycle. Avoid individual permissions where possible.
- Separate administrators, service identities, contributors, readers, auditors, and external collaborators.
- Enable least-privilege job authorization scope and restrict cross-project repository access.
- Review inactive users, PATs, OAuth grants, service hooks, extensions, agents, and service connections.

43.2 Repository and Branch Governance

Production repositories require reviewed changes, build validation, resolved comments, linked work, appropriate reviewer independence, and controlled bypass. Protect default branches and release tags. Do not treat branch policy as a substitute for secure build identity and artifact verification.

Control	Risk Reduced	Operational Detail
Minimum reviewers	Single-person error or abuse	Require independent expertise for security, network, data, and pipeline changes.
Build validation	Uncompiled or untested change	Run from trusted YAML/templates and isolate untrusted pull-request code.
Status checks	Missing external quality evidence	Fail closed for required scanners and policy engines.
Bypass control	Unreviewed emergency change	Limit to break-glass group; record reason, ticket, expiry, and retrospective.

43.3 Boards, Artifacts, and End-to-End Traceability

A release should link business requirement, threat or risk decision, source commit, build logs, tests, artifact digest, package provenance, approvals, deployment record, telemetry, and incident history. Azure Boards can provide planning traceability, while Azure Artifacts feeds can govern NuGet, npm, Maven, Python, and Universal Packages.

- Promote packages between views or feeds without rebuilding.

- Use upstream sources carefully and cache only approved ecosystems.
- Apply retention after confirming rollback and legal requirements.
- Do not use feeds as an unclassified file dump or as a secret store.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Developer cannot read repository	Group membership, project scope, repository ACL, conditional access, or guest restriction	Inspect effective permissions and Entra status; avoid granting broad project administrator access.
Branch policy is bypassed unexpectedly	Bypass permission inherited or service identity overprivileged	Audit permissions and policy history; remove inherited bypass; rotate compromised identities.
Pipeline accesses unrelated repo	Job authorization scope or explicit repository authorization is broad	Limit scope to current project/repository and explicitly authorize only required resources.
Package version disappears	Retention policy, feed view, unlisted package, or cleanup job	Inspect feed retention and recycle behavior; restore if available; align retention with rollback policy.

Hands-on Lab 43: Create a Traceable Repository and Package Workflow

Prerequisites

- Azure DevOps organization and sandbox project.
- Permission to create a repository, branch policy, board item, pipeline, and artifact feed.

Implementation

1. Create Entra-backed contributor and reviewer groups and assign minimal project permissions.
2. Create a repository with a protected main branch and CODEOWNERS-equivalent ownership documentation.
3. Configure reviewer, build-validation, comment-resolution, and work-item linking policies.
4. Commit a small package project and pipeline YAML through a pull request.
5. Publish the package to an Azure Artifacts feed using a pipeline identity.
6. Link the work item, pull request, build, package version, and release note.
7. Attempt a direct push and unauthorized package upload; capture denial evidence.

Validation Evidence

- Default branch cannot be changed directly.
- Build identity has only required repository and feed access.
- Package provenance links to the reviewed source commit.
- Bypass permissions and PAT inventory are documented.

Cleanup

- Delete test feed/package and project resources if disposable.
- Remove temporary users, groups, PATs, and service hooks.

Interview Questions and Model Answers

1. How should Azure DevOps projects be bounded?

By ownership, lifecycle, compliance, isolation, and collaboration needs rather than a fixed one-project-per-service rule.

2. Why limit job authorization scope?

It reduces the blast radius of a compromised pipeline by preventing automatic access to unrelated projects and repositories.

3. Why promote packages instead of rebuilding?

Promotion preserves artifact identity and proves the tested package is the one released.

4. What makes a release traceable?

A chain from requirement and source revision through build, tests, artifact identity, approvals, deployment, telemetry, and incidents.

Chapter Summary

- Design organization and project boundaries around least privilege and ownership.
- Protect repositories with independent review and trusted validation.
- Use immutable package versions and controlled promotion.
- Connect work, source, artifacts, releases, and operations end to end.

Chapter 44 - Azure Pipelines Continuous Integration and Agent Security

Building reproducible, isolated, credential-free CI systems with trusted templates, ephemeral agents, quality gates, and supply-chain evidence

Learning Objectives

- Design YAML pipelines with stages, templates, parameters, conditions, and deterministic artifacts.
- Secure Microsoft-hosted and self-hosted agents against untrusted code and credential theft.
- Use workload identity federation and scoped pipeline identities instead of stored cloud secrets.
- Produce tests, SBOMs, provenance, scan results, and immutable packages as CI evidence.

44.1 CI Architecture and Template Governance

A continuous-integration pipeline converts reviewed source into a verified, immutable artifact. YAML belongs in source control, but reusable security-critical logic should be centralized in protected templates. Separate compile/test/package from deployment. A build must be reproducible from the recorded source revision and dependency lock files.

```
trigger:
- main

extends:
  template: pipelines/secure-ci.yml@platform
parameters:
  language: python
  artifactName: orders-api
  runSbom: true
  failOnCritical: true
```

- Pin task major versions and review marketplace extensions.
- Use deterministic dependency resolution and trusted registries.
- Publish test results, coverage, scan reports, SBOM, provenance, and artifact digest.
- Fail closed when required controls do not run or evidence is missing.

44.2 Agent Trust Boundaries

Microsoft-hosted agents are ephemeral and reduce persistence risk. Self-hosted agents are necessary for some private networks, specialized tools, or large builds, but require patching, isolation, image management, egress control, secret cleanup, capacity, and incident procedures. Never run untrusted pull-request code on an agent that can reach production or retain sensitive credentials.

Agent Pattern	Use	Primary Controls
Microsoft-hosted	General builds with internet-reachable dependencies	No persistent state, scoped tokens, approved tasks, dependency controls.
Ephemeral self-hosted	Private network or custom toolchain	One job per instance/container, golden image, no inbound access, restricted egress.

Agent Pattern	Use	Primary Controls
Persistent self-hosted	Legacy exception only	Dedicated trust boundary, cleanup verification, patch SLA, monitoring, no cross-tenant work.

44.3 Credential-Free Build and Evidence

Azure Resource Manager service connections should use workload identity federation or managed identity where possible. The pipeline receives a short-lived token through OIDC instead of storing a client secret. Scope the federated identity and Azure role assignment narrowly, and separate build, test, publication, and deployment identities.

- Do not echo tokens, connection strings, secure variables, or full environment dumps.
- Use Key Vault only for unavoidable runtime secrets; a pipeline still needs authorization and safe masking.
- Sign artifacts or produce verifiable provenance where the delivery system supports it.
- Retain evidence according to rollback, audit, and legal requirements.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Pipeline hangs waiting for agent	No matching pool capability, agent offline, parallel-job limit, or demand mismatch	Inspect pool status, agent capabilities, queue demand, organization parallelism, and network health.
OIDC authentication fails	Issuer, subject, audience, service connection, identity, or clock mismatch	Inspect service-connection diagnostics and federated credential; verify exact subject and role assignment scope.
Secret appears in logs	Script echo, tool debug output, transformed secret not masked, or artifact upload	Revoke/rotate immediately, restrict logs, remove artifact, review access, and fix the task before rerun.
Build is not reproducible	Floating dependencies, mutable base image, environment drift, or generated untracked input	Pin locks and digests, record toolchain, isolate build, and compare artifact hashes.

Hands-on Lab 44: Build a Secure Credential-Free CI Pipeline

Prerequisites

- Azure DevOps project and protected template repository.
- A managed identity or app registration eligible for workload identity federation.
- A small application and package/image registry.

Implementation

1. Create separate build and publication identities with narrow permissions.
2. Create an Azure Resource Manager service connection using workload identity federation.
3. Build a reusable YAML template that runs lint, unit tests, dependency scan, secret scan, SBOM generation, and package publication.
4. Use a Microsoft-hosted or ephemeral self-hosted agent and document its trust boundary.

5. Publish an immutable artifact and record digest, source revision, tool versions, and evidence.
6. Run an untrusted branch scenario and prove it cannot access protected variables or production resources.
7. Revoke one permission and verify the pipeline fails with an explainable authorization error.

Validation Evidence

- No long-lived cloud secret exists in pipeline settings.
- Protected evidence is produced before artifact publication.
- Untrusted code cannot reach deployment credentials.
- Artifact identity and source revision are linked.

Cleanup

- Delete test service connection, identity assignment, packages, and agents.
- Revoke temporary tokens and remove pipeline logs containing synthetic test data.

Interview Questions and Model Answers

1. Why prefer workload identity federation?

It exchanges an OIDC assertion for short-lived Azure credentials and removes stored client-secret rotation and leakage risk.

2. When are self-hosted agents justified?

When builds need private connectivity, specialized hardware/tooling, or performance not available on hosted agents, with explicit security ownership.

3. Why separate build and deployment identities?

Compromise of build execution should not automatically grant permission to change production.

4. What evidence should CI produce?

Source revision, dependency and tool versions, tests, scans, SBOM, provenance, artifact digest, and publication record.

Chapter Summary

- CI must generate immutable, verified artifacts rather than deploy directly.
- Centralize trusted templates and isolate untrusted code.
- Use ephemeral agents and workload identity federation.
- Preserve supply-chain evidence and reproducibility.

Chapter 45 - Azure Pipelines Continuous Delivery, Environments, and Release Strategies

Controlling promotion, approvals, checks, canaries, blue-green deployment, database change, rollback, and production evidence

Learning Objectives

- Design multi-stage delivery with immutable artifact promotion and environment ownership.
- Use approvals and checks outside mutable YAML for protected resources.
- Implement rolling, canary, blue-green, and ring deployment with measurable health gates.
- Coordinate application, infrastructure, configuration, and database changes safely.

45.1 Promotion and Environment Governance

Continuous delivery promotes the same immutable artifact through environments. Azure DevOps environments provide deployment history and can own approvals and checks. Service connections, variable groups, secure files, repositories, and agent pools are protected resources and should not be broadly authorized.

- Create distinct identities and scopes for development, test, staging, and production.
- Store environment-specific nonsecret configuration separately from artifact content.
- Place production approvals, business-hour checks, branch control, policy evaluation, and exclusive locks on protected resources.
- Treat manual validation as a decision gate with evidence, not as a substitute for automated tests.

45.2 Deployment Strategies and Health Gates

Choose a strategy from workload state, traffic control, capacity, rollback speed, and dependency behavior. A deployment is successful only when technical and business signals remain healthy through an observation window.

Strategy	Best Fit	Critical Gate
Rolling	Stateless fleets with compatible versions	Capacity, readiness, error rate, and old/new compatibility.
Canary	Risky change with representative traffic	Trace, latency, error, saturation, business KPI, and abort threshold.
Blue-green	Fast switch and rollback with duplicate capacity	Data/schema compatibility, warm-up, routing, and rollback duration.
Ring	Large estates or user cohorts	Independent blast-radius progression and cohort health.

45.3 Database and Configuration Change

Use expand-and-contract database migrations: add compatible schema, deploy code that supports both versions, migrate data, verify, stop old writers, and remove obsolete schema later. Feature flags separate deployment from release but require ownership, expiry, and security review. Configuration changes deserve the same review and rollback discipline as code.

1. Validate artifact digest and environment identity.
2. Apply reversible infrastructure and additive schema changes.

3. Deploy to a small ring and validate SLO plus business signals.
4. Progress traffic or replicas only after the observation window.
5. Abort automatically when thresholds fail.
6. Record result, deployed version, approvals, exceptions, and rollback evidence.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Production stage waits indefinitely	Approval/check not completed, exclusive lock, environment permission, or protected-resource authorization	Inspect checks and resource owner; do not bypass without incident-controlled authority.
Canary looks healthy but business fails	Only infrastructure metrics monitored or cohort not representative	Add transaction, conversion, correctness, and dependency signals; replay synthetic journeys before progression.
Rollback fails	Database/schema incompatibility, irreversible config, artifact missing, or old capacity unavailable	Stop traffic progression, execute compatibility runbook, restore known-good artifact and data path, and reconcile writes.
Pipeline uses wrong subscription	Ambiguous service connection, variable, or environment mapping	Validate tenant/subscription/resource IDs in a predeployment gate and fail before change.

Hands-on Lab 45: Implement a Canary Release with External Checks

Prerequisites

- A nonproduction web workload with two versions and an observable endpoint.
- Azure DevOps environment, service connection using workload identity federation, and Azure Monitor alerts.

Implementation

1. Create build and release pipelines that promote one immutable artifact digest.
2. Create a protected staging environment and production environment with branch control and approval/check ownership.
3. Deploy the new version to a canary slot, revision, namespace, or backend pool.
4. Run smoke, synthetic, security, and compatibility tests.
5. Evaluate Azure Monitor query/alert and a synthetic business KPI before traffic progression.
6. Force an error-rate breach and verify automated abort or manual rejection.
7. Deploy a compatible version and complete promotion; record evidence and rollback target.

Validation Evidence

- Production receives only the reviewed artifact.
- Checks cannot be changed by a normal application contributor.
- Failed health gates stop progression.
- Deployment history identifies artifact, approver, identity, target, and outcome.

Cleanup

- Remove canary resources and synthetic alerts.
- Delete temporary environment permissions and test data.

Interview Questions and Model Answers

1. Why put approvals and checks outside YAML?

A contributor who can edit pipeline YAML should not be able to remove the controls protecting production resources.

2. What makes a canary valid?

Representative traffic or synthetic journeys, reliable technical and business signals, a defined observation period, and automatic abort thresholds.

3. What is expand-and-contract?

A sequence of backward-compatible schema and application changes that permits mixed-version operation and safe rollback.

4. Deployment versus release?

Deployment places capability in an environment; release exposes it to users, often through routing or feature flags.

Chapter Summary

- Promote immutable artifacts with environment-specific identity and controls.
- Use strategy-specific health gates and representative business signals.
- Design database and configuration changes for mixed-version compatibility.
- Prove abort and rollback before production.

Chapter 46 - Azure Monitor, Log Analytics, Metrics, Alerts, and KQL

Engineering a cost-aware telemetry platform with workspace architecture, diagnostic settings, data collection rules, actionable alerts, and operational dashboards

Learning Objectives

- Design Azure Monitor architecture for metrics, logs, activity, platform diagnostics, and custom telemetry.
- Use Log Analytics workspaces, data collection rules, tables, retention, and access controls intentionally.
- Write KQL for investigation, alerting, SLO analysis, and cost management.
- Create actionable alert rules, action groups, dashboards, workbooks, and runbooks.

46.1 Telemetry Architecture and Ownership

Azure Monitor combines platform metrics, logs, traces, activity records, change information, alerts, workbooks, and integrations. A monitoring platform must answer who owns each signal, what decision it drives, how quickly it must arrive, how long it is retained, and what it costs.

Signal	Use	Design Concern
Metrics	Fast numerical health and alerting	Dimensions, aggregation, cardinality, frequency, retention.
Logs	Detailed investigation and analytics	Schema, ingestion transformation, access, retention, cost.
Activity Log	Subscription control-plane events	Export continuity, tenant context, alert routing.
Resource logs	Service-specific operations	Diagnostic categories, volume, sensitive content, destination.

46.2 Workspace, DCR, and KQL Design

Use workspace topology based on data residency, access, ownership, SOC needs, and cost - not a universal one-workspace rule. Data collection rules centralize collection and transformations for supported agents and sources. Use table plans and retention according to investigation and compliance needs.

```
AzureActivity
| where TimeGenerated > ago(24h)
| where ActivityStatusValue =~ 'Failure'
| summarize Failures=count(), Operations=make_set(OperationNameValue, 5) by SubscriptionId, ResourceGroup
| order by Failures desc
```

- Filter noisy health checks and unnecessary fields before ingestion where supported.
- Protect workspace query and table access through RBAC and resource-context design.
- Create saved queries, functions, naming standards, and tests for shared KQL.
- Track ingestion by table, solution, resource, and owner; allocate or reduce cost.

46.3 Alert Engineering

An alert must represent an actionable condition with owner, severity, threshold rationale, deduplication, suppression, routing, runbook, and recovery behavior. Combine static thresholds, dynamic thresholds, log queries, activity events, service health, resource health, and synthetic tests appropriately.

- Page on symptoms that require immediate human action; ticket lower-urgency degradation.
- Alert on error-budget burn and sustained user impact rather than every transient failure.
- Test action groups and on-call routing regularly.
- Create meta-monitoring for telemetry gaps, disabled rules, failed diagnostic settings, and ingestion delay.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
No logs arrive	Diagnostic setting, DCR association, agent, endpoint, transform, table, or permission issue	Trace source to destination; inspect agent/DCR status, ingestion errors, network, and table timestamps.
Alert fires repeatedly for same incident	No suppression, flapping threshold, dimension explosion, or stateful behavior misunderstood	Tune threshold/window, group dimensions, use suppression or automation, and track incident correlation.
Query is slow or expensive	Wide time range, unbounded join, high-cardinality parsing, or missing early filter	Filter time and scope first, project needed columns, summarize early, and review query statistics.
Monitoring cost spikes	New diagnostic category, verbose app logs, cardinality, duplicate routing, or retention change	Analyze usage by table/resource, identify owner and change, filter or sample, and preserve required evidence.

Hands-on Lab 46: Build a Cost-Aware Monitoring Baseline

Prerequisites

- Sandbox subscription with Log Analytics workspace permission.
- Two test resources that expose platform metrics and resource logs.

Implementation

1. Create or select a workspace and document residency, access, retention, and cost assumptions.
2. Configure diagnostic settings or DCR-based collection for approved categories only.
3. Create KQL queries for failures, latency, top talkers, ingestion volume, and missing heartbeat/telemetry.
4. Create metric and log alerts with severity, owner, runbook, and action group.
5. Build a workbook showing service health, release markers, error trend, and cost/ingestion.
6. Generate a controlled failure and prove alert delivery and recovery.
7. Disable one diagnostic path and verify a meta-monitor detects the gap.

Validation Evidence

- Telemetry arrives with expected schema and delay.
- Alert notifications contain useful context and ownership.
- KQL queries are bounded and reusable.
- Ingestion volume and retention have an accountable owner.

Cleanup

- Delete test alerts, action groups, workbooks, and diagnostic settings.

- Remove synthetic logs and temporary permissions.

Interview Questions and Model Answers

1. Metrics versus logs?

Metrics are optimized numerical time series for fast monitoring; logs provide rich records for flexible analysis and investigation.

2. What is a DCR?

A Data Collection Rule defines supported data sources, transformations, destinations, and collection behavior.

3. Why do alerts need runbooks?

A notification without a decision path, evidence, owner, and safe actions increases noise rather than reducing recovery time.

4. How do you control Azure Monitor cost?

Collect only useful data, transform/filter early, choose table plans and retention intentionally, manage cardinality, and allocate usage to owners.

Chapter Summary

- Design telemetry from decisions, ownership, and cost.
- Use DCRs, workspaces, KQL, and access controls intentionally.
- Create symptom- and SLO-based actionable alerts.
- Monitor the monitoring system itself.

Chapter 47 - Application Insights, OpenTelemetry, SLOs, and Distributed Tracing

Connecting user experience, service dependencies, releases, traces, logs, metrics, availability, and error budgets

Learning Objectives

- Instrument applications with OpenTelemetry and Azure Monitor Application Insights.
- Design trace propagation, resource attributes, sampling, filtering, and sensitive-data controls.
- Define SLIs, SLOs, error budgets, burn-rate alerts, and release gates.
- Troubleshoot latency, dependency failures, sampling gaps, and broken correlation.

47.1 OpenTelemetry and Application Insights

Application Insights is the application-performance monitoring capability of Azure Monitor. OpenTelemetry provides vendor-neutral APIs, SDKs, and semantic conventions for traces, metrics, and logs. Use the Azure Monitor OpenTelemetry distribution where supported, and keep instrumentation separate from business logic.

- Set service name, service version, deployment environment, region, and instance attributes consistently.
- Propagate W3C trace context across HTTP, messaging, events, workflows, and background jobs.
- Capture dependency duration, result, retry count, queue wait, and business outcome without recording secrets or regulated payloads.
- Use sampling deliberately and preserve high-value failures and rare paths.

47.2 SLI, SLO, and Error-Budget Engineering

An SLI is a measured service indicator such as successful request ratio or latency. An SLO is a target over a window. The error budget is the allowable unreliability. SLOs should reflect user journeys and service commitments, not only CPU or host availability.

Journey	Example SLI	SLO Decision
Order API	Valid requests completed successfully	99.9% over 28 days excluding agreed invalid input.
Checkout	End-to-end completion latency	99% under 2 seconds for eligible traffic.
Async fulfillment	Accepted orders completed within deadline	99.5% within 10 minutes.
Data pipeline	Freshness of validated dataset	99% available by agreed business time.

- Use fast and slow burn-rate alerts to detect severe and sustained error-budget consumption.
- Freeze risky releases or invest in reliability when budget policy is exceeded.
- Exclude traffic only through documented, reviewed rules.
- Publish SLO ownership and data-quality checks.

47.3 Trace-Based Investigation and Release Correlation

A distributed trace should connect edge request, API gateway, service, database, queue, function, workflow, and external dependency. Add deployment version and feature-flag attributes so operators can compare before and after a release. Use transaction search, application maps, KQL, and trace exemplars to move from symptom to root cause.

```
requests
| where timestamp > ago(2h)
| summarize total=count(), failed=countif(success == false), p95=percentile(duration,95) by cloud_RoleName,
bin(timestamp,5m)
| extend failureRate = todouble(failed) / total
| order by timestamp desc
```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Trace stops at queue	Trace context not injected/extracted or message attributes lost	Inspect producer and consumer instrumentation; standardize context fields and test duplicate/retry paths.
Telemetry contains sensitive data	Automatic collection, custom attributes, request body, query string, or exception content	Restrict access, purge where legally supported, rotate exposed credentials, filter at source, and add tests.
Latency high but service spans look normal	Edge, queue wait, DNS, client, dependency, sampling, or missing span	Compare end-to-end SLI with trace duration, inspect queue and client timing, and add missing instrumentation.
SLO dashboard is misleading	Wrong denominator, excluded failures, ingestion delay, sampling, or query drift	Validate against raw transactions, version queries, test edge cases, and assign SLO data owner.

Hands-on Lab 47: Instrument a Distributed Transaction and Create an SLO Gate

Prerequisites

- A small API plus background consumer or function.
- Application Insights linked to a Log Analytics workspace.
- Ability to deploy OpenTelemetry configuration and Azure Monitor alerts.

Implementation

1. Instrument API, dependency, and worker using OpenTelemetry with standard resource attributes.
2. Propagate trace context through an HTTP call and a queue/message.
3. Add deployment version and synthetic business transaction ID without sensitive payloads.
4. Create an availability or synthetic test for the critical journey.
5. Define success-rate and latency SLIs, a 28-day SLO, and fast/slow burn queries.
6. Deploy a slow/failing version and prove trace correlation and release marker identify impact.
7. Use the SLO signal as a nonproduction release gate and document production policy.

Validation Evidence

- One trace spans all transaction components.
- Sensitive values are absent or filtered.
- SLO query denominator and exclusions are reviewed.
- Release gate blocks a deliberately unhealthy version.

Cleanup

- Delete test telemetry resources or reduce retention after evidence capture.
- Remove synthetic endpoints, queues, and temporary alerts.

Interview Questions and Model Answers

1. Why OpenTelemetry?

It provides standardized, vendor-neutral instrumentation and context propagation while Azure Monitor supplies collection and analysis experiences.

2. What is an error budget?

The amount of unreliability permitted by an SLO during its measurement window.

3. Why instrument queue wait?

Async latency often occurs before processing starts; service execution spans alone can hide user-visible delay.

4. How should sampling be designed?

Balance cost and statistical accuracy while preserving failures, high-latency paths, critical transactions, and cross-service correlation.

Chapter Summary

- Use OpenTelemetry to create consistent cross-platform telemetry.
- Design SLOs from user journeys and reliable denominators.
- Correlate traces with release versions and business outcomes.
- Filter sensitive data at the source and validate telemetry quality.

Chapter 48 - Azure Audit, Resource Graph, Automation, and Fleet Operations

Operating the Azure estate through Activity Log, change evidence, Resource Graph, Automation, Update Manager, Arc, and controlled runbooks

Learning Objectives

- Build a durable control-plane audit and change-investigation capability.
- Use Azure Resource Graph for inventory, policy, configuration, and drift queries at scale.
- Design secure automation with managed identities, runbook approvals, and idempotent operations.
- Operate hybrid and Azure fleets through Azure Arc, Update Manager, maintenance controls, and evidence.

48.1 Audit and Change Evidence

The Azure Activity Log records subscription control-plane operations. Export it to durable destinations for investigation and retention. Combine activity, resource logs, Entra sign-in/audit data, pipeline history, policy events, and deployment operations to answer who changed what, from where, with which identity, and with what result.

- Alert on policy assignment, role assignment, lock, network, Key Vault, diagnostic-setting, and security-control changes.
- Preserve tenant, subscription, correlation ID, caller, operation, status, resource ID, and timestamp.
- Use change-analysis capabilities and deployment history to compare configuration before and after incidents.
- Do not rely on portal history alone for regulated retention.

48.2 Resource Graph and Estate Inventory

Azure Resource Graph queries resources across subscriptions quickly using KQL-like syntax. Use it for inventory, tag compliance, public exposure, outdated images, missing diagnostics, unsupported regions, and ownership gaps. It is not a replacement for service telemetry or a configuration database with transactional guarantees.

```
Resources
| where type =~ 'microsoft.compute/virtualmachines'
| extend owner=tostring(tags.owner), env=tostring(tags.environment)
| project subscriptionId, resourceGroup, name, location, owner, env
| order by subscriptionId, resourceGroup
```

48.3 Automation and Fleet Operations

Azure Automation runbooks, Functions, Logic Apps, Azure DevOps, and GitHub Actions can execute operational workflows. Choose a system with clear source control, identity, approval, retry, logging, and ownership. Azure Update Manager coordinates update assessment and scheduling for supported Azure and Arc-enabled machines.

Operation	Control
Patch fleet	Maintenance window, classification, precheck, ring, health gate, rollback/recovery, evidence.
Remediate policy	Narrow scope, managed identity, preview, idempotency, throttling, exception handling.

Operation	Control
Rotate certificate	Inventory, dependency map, overlap period, validation, expiry alert.
Stop idle resource	Ownership confirmation, dependency and data check, schedule, exception, re-enable path.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Change actor is unclear	Shared identity, missing exported logs, token reuse, or insufficient correlation	Correlate activity, Entra, pipeline, deployment, and agent logs; eliminate shared credentials and improve identity attribution.
Resource Graph query misses resource	Scope, permission, propagation delay, unsupported property, or wrong type casing/path	Verify accessible subscriptions and raw resource shape; compare with ARM API and allow for indexing delay.
Automation repeats destructive action	Non-idempotent script, retry, duplicate trigger, or missing state check	Disable trigger, inspect execution history, add preconditions and idempotency key, and require approval for destructive scope.
Patch ring causes outage	Inadequate capacity, dependency order, health gate, reboot behavior, or maintenance conflict	Stop rollout, restore capacity, validate service health, revise ring size and pre/post checks.

Hands-on Lab 48: Create an Azure Estate Inventory and Safe Remediation Runbook

Prerequisites

- Two sandbox subscriptions or resource groups with sample resources.
- Log Analytics workspace and permission to query Resource Graph.
- Managed identity for automation.

Implementation

1. Export Activity Log to a controlled workspace or storage destination.
2. Create Resource Graph queries for missing owner tags, public IPs, missing diagnostics, and unattached resources.
3. Build an idempotent runbook that adds a missing nonsecurity tag to an approved test scope.
4. Authenticate the runbook with managed identity and narrow role assignment.
5. Add dry-run mode, scope allowlist, throttling, change ticket input, and structured logs.
6. Execute against a small ring and verify no unrelated resources change.
7. Create an alert for unauthorized role or diagnostic-setting changes.

Validation Evidence

- Inventory results are reproducible and scoped.
- Runbook logs identity, target, before/after state, and result.

- Repeated execution produces no unintended change.
- Audit alert reaches an owned action group.

Cleanup

- Remove test automation account/runbook and role assignment.
- Delete temporary resources and retain only approved evidence.

Interview Questions and Model Answers

1. What is Azure Resource Graph best for?

Fast cross-subscription inventory and configuration queries across Azure resource metadata.

2. Why export Activity Log?

Portal retention and access are not a complete long-term audit strategy; export supports centralized investigation and required retention.

3. What makes a remediation safe?

Narrow scope, dry run, explicit identity, idempotency, rate limiting, preconditions, evidence, and rollback or recovery.

4. How should patching be rolled out?

Through small rings with capacity checks, maintenance coordination, health gates, and pause/rollback authority.

Chapter Summary

- Centralize control-plane evidence and identity attribution.
- Use Resource Graph for estate-scale inventory and drift analysis.
- Build source-controlled, idempotent automation with managed identity.
- Operate fleets through rings, health gates, and auditable maintenance.

Chapter 49 - Azure Key Vault, Managed HSM, Secrets, Keys, and Certificates

Building protected cryptographic and secret lifecycles with identity-based access, private networking, rotation, recovery, and dependency-aware operations

Learning Objectives

- Choose Key Vault or Managed HSM according to key ownership, isolation, compliance, and cryptographic requirements.
- Design RBAC, managed identity, private access, logging, purge protection, and separation of duties.
- Implement secret, key, and certificate rotation with overlap and dependency validation.
- Respond to exposure, deletion, expiry, access-denial, throttling, and network failures.

49.1 Vault Architecture and Access Model

Azure Key Vault stores secrets, keys, and certificates; Managed HSM provides a single-tenant managed HSM service for cryptographic keys. Use separate vaults when workload, environment, region, ownership, or compliance boundaries require them. Prefer Azure RBAC and managed identities, and avoid giving application operators broad secret-read access.

Object	Typical Use	Control Focus
Secret	Connection data or unavoidable credential	Minimize use, rotate, version, audit, no log exposure.
Key	Encryption/signing key	Algorithm, HSM requirement, rotation, backup/recovery, separation.
Certificate	TLS or signing certificate lifecycle	Issuer, renewal, overlap, dependency deployment, expiry alert.
Managed HSM key	High-assurance cryptographic control	Security domain, quorum/recovery, role separation, region design.

49.2 Network, Recovery, and Rotation

Use private endpoints and private DNS when the workload network model requires private access. Firewall settings and trusted-service exceptions must be explicit. Enable soft delete and purge protection according to policy. A rotation design must identify all consumers, cache behavior, version pinning, overlap, failure rollback, and emergency revocation.

- Applications should retrieve secrets at runtime through managed identity or use secretless service authentication.
- Do not put secret values in pipeline outputs, ARM/Bicep outputs, Terraform state, application logs, crash dumps, or support tickets.
- Alert on near-expiry certificates, access-policy/RBAC changes, purge attempts, disabled keys, and unusual secret access.
- Test recovery in a sandbox, including DNS and identity dependencies.

49.3 Cryptographic Operations and Incident Response

Key rotation is not complete until encrypted or signed consumers can use the new key and still process required old data. Envelope encryption separates data-encryption keys from key-encryption keys. During exposure, identify versions, consumers, access logs, data impact, and replacement order before disabling credentials blindly.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Application receives 403	Wrong identity, RBAC scope, token audience, network, disabled object, or deny policy	Confirm runtime principal and token, effective role, vault firewall/private DNS, object state, and diagnostic logs.
Certificate did not renew	Issuer/contact/configuration, permission, unsupported binding, or dependency deployment failure	Inspect certificate operation and event; renew manually if needed; validate every endpoint before removing old version.
Vault name resolves publicly	Private DNS link/zone record missing or client using external resolver	Resolve from the runtime network, inspect private endpoint and DNS forwarding, and block public access only after validation.
Secret was exposed	Log or repository leak, broad access, compromised identity, or support artifact	Revoke/rotate, remove exposure, preserve evidence, investigate access, notify owners, and prevent recurrence.

Hands-on Lab 49: Deploy a Private Key Vault with Rotation Evidence

Prerequisites

- Sandbox VNet and private DNS permissions.
- Managed identity for a small test application or VM.

Implementation

1. Create a Key Vault with soft delete, purge protection, RBAC, logging, and secure public-network setting.
2. Create a private endpoint and validate private DNS from the authorized runtime.
3. Assign the runtime identity permission to read one test secret only.
4. Retrieve the secret at runtime without storing it in code or pipeline variables.
5. Create a second secret version and implement overlap-aware rotation.
6. Attempt unauthorized list/read/purge operations and capture denial evidence.
7. Create expiry/access-change alerts and a recovery runbook.

Validation Evidence

- Authorized runtime resolves and accesses the vault privately.
- Unauthorized identity and network paths are denied.
- Rotation completes without exposing values.
- Audit logs show principal, operation, result, and object metadata.

Cleanup

- Delete test application and private endpoint after recovery-policy review.
- Remove test secrets, role assignments, DNS records, and alerts using approved purge rules.

Interview Questions and Model Answers

1. Key Vault versus Managed HSM?

Key Vault provides managed secrets, keys, and certificates; Managed HSM provides single-tenant managed HSM-backed key storage for stronger isolation and cryptographic-control needs.

2. Why prefer managed identity?

It removes stored application credentials and lets Azure issue short-lived tokens to a known workload identity.

3. Why can rotation cause outages?

Consumers may cache old values, pin versions, require synchronized deployment, or be unable to decrypt data created with previous keys.

4. What do soft delete and purge protection provide?

Recovery against accidental or malicious deletion by retaining deleted objects and preventing early permanent purge according to policy.

Chapter Summary

- Use workload-specific vault boundaries and least-privilege managed identity.
- Design private networking, logging, soft delete, and purge protection together.
- Treat rotation as a dependency-aware production change.
- Respond to exposure with evidence, revocation, and root-cause remediation.

Chapter 50 - Microsoft Defender for Cloud and Cloud Security Posture Management

Operating posture, regulatory compliance, attack-path analysis, vulnerability management, workload protection, and security recommendations at enterprise scale

Learning Objectives

- Explain Defender for Cloud posture management and workload-protection capabilities.
- Prioritize recommendations using asset criticality, exposure, attack paths, exploitability, and business context.
- Connect policy, secure score, compliance, vulnerability findings, and remediation ownership.
- Manage exemptions, suppressions, connectors, plans, and security data with governance and cost control.

50.1 Posture Management versus Workload Protection

Cloud security posture management identifies misconfiguration, exposure, governance gaps, and attack paths. Cloud workload protection plans provide service-specific threat detection and protection capabilities. A high secure score is useful direction, not proof of security or compliance.

- Onboard subscriptions through management-group policy and confirm plan, agentless scanning, extensions, and data settings.
- Assign recommendations to accountable teams with due dates and exception paths.
- Use asset criticality and internet exposure to prioritize paths that lead to sensitive data or privileged control.
- Validate remediation in lower environments; security fixes can affect connectivity and availability.

50.2 Recommendation Lifecycle and Compliance Evidence

A recommendation becomes useful only when it has owner, severity, affected assets, risk rationale, remediation, validation, exception criteria, and closure evidence. Regulatory dashboards map technical assessments to controls, but governance teams must validate applicability and evidence quality.

Disposition	Required Evidence
Remediate	Change record, deployed fix, rescan, workload validation, closure owner.
Exempt temporarily	Business justification, compensating control, approver, scope, expiry, review date.
Accept risk	Named accountable executive, impact, duration, monitoring, and reconsideration trigger.
False positive	Technical proof, product/version context, Microsoft or scanner reference, review.

50.3 Vulnerability and Attack-Path Operations

Integrate vulnerability management for machines, registries, containers, databases, and cloud resources where supported. Deduplicate findings, correlate with deployment and ownership, and prioritize reachable vulnerabilities in critical assets. An attack path is a hypothesis that must be investigated and broken at the most effective control point.

- Set remediation SLAs by exploitability, exposure, data sensitivity, and service criticality.

- Patch or rebuild immutable workloads rather than modifying unmanaged instances when possible.
- Track mean time to assign, remediate, validate, and close.
- Monitor disabled plans, stale connectors, scan gaps, and recommendation suppression.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Subscription shows no recommendations	Plan/initiative not assigned, connector disabled, scan delay, unsupported resource, or permission	Inspect Defender settings, policy compliance, extensions/connectors, resource type, and onboarding status.
Secure score falls after onboarding	New assessments or assets changed denominator	Review recommendation changes and risk; do not game score by disabling plans or exemptions.
Remediation breaks workload	Security recommendation applied without network/application test	Roll back safely, restore service, evaluate compensating control, and stage corrected remediation.
Vulnerability remains after patch	Image not rebuilt, stale scanner, wrong instance, package not removed, or runtime drift	Verify deployed digest/package inventory, rescan timing, and whether old replicas remain.

Hands-on Lab 50: Build a Defender for Cloud Recommendation Workflow

Prerequisites

- Sandbox subscription with Defender for Cloud access.
- A deliberately misconfigured test resource that does not contain real data.

Implementation

1. Review Defender plans, pricing implications, and policy assignments before enabling required lab capabilities.
2. Inventory recommendations and group by owner, criticality, exposure, and remediation type.
3. Select one safe recommendation and document risk, change, validation, and rollback.
4. Remediate through Bicep or controlled configuration rather than portal-only change.
5. Wait for reassessment and capture closure evidence.
6. Create a temporary exemption for a second synthetic finding with owner and expiry, then remove it.
7. Document attack-path, vulnerability, and plan-health monitoring.

Validation Evidence

- Recommendation ownership and risk rationale are explicit.
- Remediation is reproducible and workload validation passes.
- Exemption is narrow, approved, and expires.
- Cost and data-collection implications are documented.

Cleanup

- Delete the intentionally vulnerable resource.
- Disable paid lab plans only if organizational policy permits and after evidence capture.

Interview Questions and Model Answers

1. What is CSPM?

Continuous identification and prioritization of cloud configuration, exposure, governance, and attack-path risk.

2. Does secure score prove compliance?

No. It summarizes selected recommendations; compliance requires applicable control design, implementation, operation, and evidence.

3. How should recommendations be prioritized?

By exploitability, reachability, asset criticality, data sensitivity, exposure, attack path, and business impact.

4. What makes an exemption acceptable?

Narrow scope, documented risk and compensating controls, accountable approval, expiry, monitoring, and periodic review.

Chapter Summary

- Use Defender for Cloud as an operational risk system, not a score dashboard.
- Prioritize attack paths and reachable critical assets.
- Make remediation and exceptions owned, time-bound, and evidenced.
- Monitor coverage, scanning, plan health, and cost.

Chapter 51 - Microsoft Sentinel and Security Operations Engineering

Designing SIEM/SOAR ingestion, analytics, incidents, hunting, automation, retention, and SOC operating processes in the Microsoft security ecosystem

Learning Objectives

- Design Sentinel workspace, data-connector, table, retention, and access architecture.
- Build analytics rules with entity mapping, tuning, suppression, and measurable quality.
- Create incident triage, investigation, hunting, automation, containment, and recovery playbooks.
- Control security-data volume, privacy, evidence integrity, and SOC operating cost.

51.1 SIEM/SOAR Architecture and Data Quality

Microsoft Sentinel analyzes security data in Log Analytics and is increasingly operated through the Microsoft Defender portal ecosystem. Connector design begins with threats and use cases, not with ingesting every available log. Each source needs owner, schema, latency, retention, cost, health monitoring, and privacy classification.

Source	Security Value	Quality Check
Microsoft Entra	Authentication, privilege, application, risk	Tenant, identity, conditional-access result, ingestion latency.
Azure Activity	Control-plane and policy changes	Caller, correlation ID, operation, scope, status.
Defender alerts	Workload and posture detections	Alert mapping, severity, asset identity, duplication.
Network/security devices	Traffic and enforcement evidence	Time sync, parser/schema, source coverage, dropped events.

51.2 Analytics, Incidents, and Automation

Analytics rules convert telemetry into alerts and incidents. Tune rules using known attack techniques, entity mapping, thresholds, suppression, watchlists, threat intelligence, and environment context. Automation rules and playbooks can enrich, notify, ticket, or contain, but destructive response requires strong authorization and safety checks.

- Measure true-positive rate, false-positive rate, time to triage, time to contain, and coverage of priority techniques.
- Group related alerts by entity and time to reduce analyst noise.
- Automate reversible enrichment first; automate containment only after testing and approval.
- Preserve original evidence and chain-of-custody requirements.

51.3 Hunting and Incident Lifecycle

Threat hunting is hypothesis-driven analysis that can produce new detections, watchlists, and response procedures. An incident lifecycle includes preparation, detection, triage, scoping, containment, eradication, recovery, communications, evidence handling, and lessons learned.

```

SigninLogs
| where TimeGenerated > ago(24h)
| where ResultType != 0
| summarize Failures=count(), IPs=dcount(IPAddress), Apps=make_set(AppDisplayName,5) by UserPrincipalName
| where Failures > 20
| order by Failures desc

```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Connector health is green but data missing	Wrong table, source-side filter, tenant/workspace mismatch, transform, license, or parsing issue	Compare source counts with table timestamps and schema; test a known event end to end.
Rule creates excessive incidents	Threshold too low, missing entity grouping, duplicate sources, or normal behavior	Analyze samples, add context and grouping, tune suppression, and document accepted residual risk.
Playbook cannot contain account	Managed identity lacks role, connector authorization expired, or conditional access	Inspect run history and principal; fix narrow permission and retest in a simulation.
SOC cost increases sharply	New connector, verbose table, duplicate ingestion, long retention, or high query frequency	Analyze usage by table/source, filter before ingestion, adjust retention/tier, and preserve critical evidence.

Hands-on Lab 51: Create a Sentinel Detection and Safe Automation Playbook

Prerequisites

- Sentinel-enabled workspace or approved lab environment.
- Synthetic identity or resource events and permission to create analytics/automation rules.

Implementation

1. Define one threat hypothesis and required data sources.
2. Validate connector health, event timestamp, entity fields, and expected latency.
3. Write and test a KQL detection using a historical/synthetic dataset.
4. Create an analytics rule with entity mapping, grouping, severity, and suppression.
5. Generate a controlled event and triage the resulting incident.
6. Create a playbook that enriches and opens a ticket or sends an owned notification using managed identity.
7. Document conditions required before any automated containment and run a tabletop exercise.

Validation Evidence

- Rule detects the intended event with acceptable noise.
- Incident contains useful entities and investigation context.
- Automation is reversible, least privileged, and audited.
- Detection, response, and data-cost metrics are recorded.

Cleanup

- Remove synthetic incidents, watchlists, analytics rules, and playbooks.

- Revoke temporary identity permissions and delete test data where allowed.

Interview Questions and Model Answers

1. What is the difference between an alert and an incident?

An alert is a detection result; an incident groups alerts and entities into a case for investigation and response.

2. How do you tune a detection?

Validate data, analyze true and false positives, add context/entity mapping, adjust thresholds/windows/grouping, and document residual risk.

3. Why not automate all containment?

Incorrect containment can cause business outage or destroy evidence; automate only tested, authorized, reversible actions with guardrails.

4. How do you control Sentinel cost?

Prioritize security use cases, filter at source, avoid duplicates, choose retention/tier intentionally, and monitor ingestion by source and owner.

Chapter Summary

- Design SIEM ingestion from threat use cases and data quality.
- Tune analytics for measurable signal quality.
- Use safe, identity-based automation and preserve evidence.
- Run a complete incident lifecycle and feed lessons into controls.

Chapter 52 - Azure Network Protection and Zero Trust Enforcement

Combining NSGs, Azure Firewall, WAF, DDoS Protection, Private Link, DNS, routing, segmentation, and centralized policy without creating hidden failure domains

Learning Objectives

- Design layered network controls from explicit application flows and Zero Trust principles.
- Select NSGs, Azure Firewall, WAF, DDoS Protection, Private Link, and routing services appropriately.
- Centralize policy and logging while preserving regional resilience and application ownership.
- Troubleshoot asymmetric routing, DNS, SNAT, TLS, WAF, firewall, and private-endpoint failures.

52.1 Layered Enforcement Model

No single control provides complete network security. NSGs filter subnet/NIC flows; Azure Firewall provides centralized network and application policy; WAF protects HTTP applications; DDoS Protection addresses volumetric attacks; Private Link exposes PaaS services through private endpoints; routing and DNS determine the actual path.

Layer	Control Question
Identity	Which user or workload is calling, and is the token authorized?
Edge	Is the public request protected by TLS, WAF, rate limits, and DDoS controls?
Network	Is the source, destination, port, protocol, route, and egress allowed?
Service	Does the target accept this identity, network, and operation?
Data	Is the requested data permitted, encrypted, logged, and minimized?

52.2 Hub-Spoke and Egress Security

A hub-spoke or Virtual WAN topology can centralize connectivity, DNS, firewall, and inspection. Centralization increases dependency on route correctness, capacity, regional design, and change governance. Forced tunneling and user-defined routes must be tested for Azure platform dependencies and asymmetric paths.

- Use application and network rule collections with explicit owners, source/destination, purpose, and expiry.
- Prefer service tags and FQDN controls only when their semantics match the workload.
- Monitor SNAT port consumption, firewall health, latency, rule hits, threat-intelligence actions, and denied traffic.
- Separate emergency access from normal administration and use Bastion or private management paths.

52.3 Private Link, DNS, and WAF Operations

Private endpoints change service access to private IPs but do not automatically configure every DNS client or disable public access. Design private DNS zones, links, forwarding, split-horizon behavior, on-premises resolution, and failover. WAF rules require tuning against real application behavior and should be deployed in detection mode before prevention where appropriate.

- Validate TLS hostname and certificate after private DNS changes.

- Create synthetic tests from each trust zone.
- Treat WAF exclusions as narrow, documented, and expiring.
- Log enough request context to investigate without storing sensitive payloads.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Private endpoint exists but app cannot connect	DNS, route, NSG, firewall, target approval, public-access setting, or TLS name mismatch	Resolve from runtime, trace route, inspect flow/firewall logs, validate endpoint state and hostname.
Firewall allows one direction only	Asymmetric routing, missing return route, NAT, or state mismatch	Inspect effective routes at both ends and appliance/firewall connection logs; correct topology before adding broad rules.
WAF blocks valid traffic	Managed rule false positive, body encoding, size, bot rule, or custom rule order	Analyze rule ID and sample request, create narrow exclusion or app fix, test detection mode, and set expiry.
Outbound calls fail intermittently	SNAT exhaustion, DNS timeout, dependency throttling, firewall scaling, or route change	Inspect SNAT metrics, connection reuse, resolver health, dependency metrics, and recent route/rule changes.

Hands-on Lab 52: Build a Private Application Path with Central Inspection

Prerequisites

- Sandbox hub and spoke VNETs or equivalent small topology.
- A test web endpoint, Key Vault/storage private endpoint, Azure Firewall or documented simulation alternative.

Implementation

1. Draw the required client-to-edge-to-app-to-data flows and expected DNS names.
2. Create spoke segmentation and narrow NSG rules.
3. Route approved egress through a central firewall and create explicit application/network rules.
4. Create a private endpoint and private DNS zone; validate resolution from spoke and an unauthorized network.
5. Place a WAF-capable edge in detection mode for a synthetic web endpoint if budget permits.
6. Generate allowed and denied traffic and correlate NSG, firewall, WAF, and service logs.
7. Document regional failure, DNS failure, and emergency-access procedures.

Validation Evidence

- Authorized flow succeeds through expected controls.
- Unauthorized source and public service path are denied as designed.
- DNS returns correct private/public answers by trust zone.
- Logs identify the enforcing control and rule.

Cleanup

- Delete premium network resources promptly after evidence capture.

- Remove UDRs, private endpoints, DNS links, and temporary firewall/WAF rules.

Interview Questions and Model Answers

1. Private endpoint versus service endpoint?

Private endpoint assigns a private IP representing the service in the VNet; service endpoint extends VNet identity to the service public endpoint.

2. Why is DNS central to Private Link?

Clients must resolve the service hostname to the private endpoint while preserving TLS hostname and connectivity from every required network.

3. What does WAF protect?

HTTP/S application traffic against common web attacks and custom policy conditions; it is not a replacement for application authorization or secure code.

4. Why can centralized firewalls become a reliability risk?

Routing, capacity, regional dependency, policy errors, or maintenance can affect many spokes simultaneously.

Chapter Summary

- Build layered controls from explicit flows and identity.
- Centralize network enforcement with capacity and failure-domain awareness.
- Design Private Link and DNS together.
- Tune WAF and firewall policy with evidence and expiring exceptions.

Chapter 53 - Azure Landing Zones, Policy, and Governance at Scale

Establishing management-group hierarchy, subscription vending, identity, connectivity, management, security, policy, and compliance evidence for enterprise Azure adoption

Learning Objectives

- Explain platform and application landing zones and their design areas.
- Design management-group hierarchy, subscription vending, policy initiatives, role assignments, and exceptions.
- Implement governance as code using Bicep or approved landing-zone accelerators.
- Measure policy compliance, remediation, drift, ownership, and control effectiveness.

53.1 Landing-Zone Operating Model

An Azure landing zone is the recommended standardized approach for operating Azure at scale. Platform landing zones provide shared identity, connectivity, management, and governance. Application landing zones host workload subscriptions. The hierarchy must reflect policy inheritance and operating responsibility, not the organization chart alone.

Design Area	Enterprise Decision
Tenant and billing	Directory ownership, enrollment, subscription creation, cost allocation.
Identity and access	Administrative model, PIM, break glass, workload identity, external access.
Resource organization	Management groups, subscription archetypes, naming, tags, ownership.
Network topology	Hub-spoke/Virtual WAN, DNS, egress, private connectivity, regional pattern.
Management and security	Logging, monitoring, Defender, Sentinel, backup, patching, incident response.
Governance	Policy, compliance, exemptions, remediation, evidence, sovereign requirements.

53.2 Policy and Subscription Vending

Azure Policy initiatives enforce or audit standards across management-group scope. Effects such as deny, modify, deployIfNotExists, and auditIfNotExists have different risks and remediation requirements. Subscription vending should create subscriptions with hierarchy placement, budgets, policy, networking, logging, identity, owner, and lifecycle metadata automatically.

- Test policy in audit mode and lower scopes before deny at scale.
- Use managed identities for remediation and grant only required roles.
- Version custom definitions and initiatives as code.
- Create exemption workflow with owner, justification, scope, compensating control, expiry, and review.

53.3 Governance Evidence and Drift

Compliance percentage is not the same as control effectiveness. Validate that policies cover intended resource types, evaluate on time, remediate correctly, and cannot be bypassed by alternate services or scopes. Use Resource Graph, Policy Insights, deployment history, Activity Log, and security tools to detect drift.

1. Define control objective and accountable owner.
2. Implement policy and platform capability as code.
3. Pilot at a narrow scope.
4. Measure false positives, workload impact, and remediation identity.
5. Promote through management groups.
6. Review exemptions and noncompliance.
7. Test the control through a realistic violation.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Policy assignment has no effect	Wrong scope, exclusion, mode, alias, resource type, enforcement mode, or evaluation delay	Inspect assignment and definition, effective scope, compliance reason, and a known test resource.
DeployIfNotExists remediation fails	Managed identity missing role, template invalid, region/resource conflict, or policy condition	Inspect remediation task and deployment operations; fix narrow role and template.
Subscription lands in wrong hierarchy	Vending workflow mapping, manual move, permission, or management-group change	Stop workload onboarding, move through controlled process, re-evaluate policy and RBAC, and audit impact.
Deny policy blocks emergency repair	Overbroad rule or no break-glass process	Use controlled exemption with expiry and incident evidence; correct policy after service restoration.

Hands-on Lab 53: Implement a Policy-Driven Application Landing Zone

Prerequisites

- Sandbox management group or subscription scope.
- Permission to assign a custom policy at a safe scope and create a test subscription/resource group.

Implementation

1. Define a simple application-landing-zone contract: owner tags, allowed regions, diagnostics, public network rule, budget, and identity baseline.
2. Create policy definitions/initiative as code and deploy first with audit effects.
3. Create compliant and noncompliant test resources and inspect compliance reasons.
4. Add a remediation identity for one safe deployIfNotExists or modify policy and run a task.
5. Promote one tested rule to deny at a narrow scope.
6. Create and expire a documented exemption.
7. Build a governance workbook showing compliance, exemptions, missing owners, and remediation failures.

Validation Evidence

- Policy behavior matches the control objective.
- Remediation identity has minimal rights.
- Deny effect blocks only the intended violation.
- Exemption and evidence are owned and time-bound.

Cleanup

- Remove test policy assignments, exemptions, and resources.
- Retain definitions and test evidence in source control if approved.

Interview Questions and Model Answers

1. Platform versus application landing zone?

Platform landing zones host shared services; application landing zones are workload subscriptions operating within inherited guardrails.

2. Why not mirror the organization chart in management groups?

Management groups primarily drive policy and access inheritance; frequent organizational changes create brittle governance.

3. Audit versus deny?

Audit reports noncompliance; deny prevents creation/update. Deny requires stronger testing and exception procedures.

4. What makes policy-as-code effective?

Versioned definitions, tests, staged rollout, narrow remediation identity, compliance evidence, and managed exemptions.

Chapter Summary

- Use landing zones to standardize enterprise Azure foundations.
- Design hierarchy for policy and responsibility.
- Automate subscription vending and governance as code.
- Measure control effectiveness, drift, remediation, and exemptions.

Chapter 54 - Azure Well-Architected, Resilience Engineering, and FinOps

Balancing reliability, security, cost, operations, and performance through explicit requirements, failure testing, capacity models, and financial accountability

Learning Objectives

- Apply Azure Well-Architected pillars to workload decisions and trade-offs.
- Define RTO, RPO, SLO, dependency, capacity, and recovery requirements.
- Run failure-mode analysis, game days, backup/restore, failover, and chaos experiments safely.
- Implement FinOps allocation, budgets, forecasting, unit economics, commitments, and optimization governance.

54.1 Well-Architected Trade-offs

The Azure Well-Architected Framework evaluates reliability, security, cost optimization, operational excellence, and performance efficiency. Improvements in one pillar can affect another: duplicated regional capacity increases cost; aggressive cost reduction can reduce recovery capacity; deep security inspection can add latency; excessive telemetry raises cost and privacy risk.

Decision	Trade-off Questions
Multi-region active-active	Does business value justify data consistency, capacity, operations, and testing complexity?
Private-only service	How will pipelines, support, DNS, failover, and third parties connect?
Autoscale to zero	Is cold-start latency compatible with SLO and dependency behavior?
Long log retention	What investigation/compliance value justifies ingestion and retention cost?

54.2 Resilience Engineering

Start with business impact analysis and dependency mapping. RTO is the target time to restore service; RPO is acceptable data loss measured in time. Backups do not prove recovery until restored and validated. Regional resilience requires traffic, identity, DNS, keys, secrets, network, data, messages, observability, capacity, and operator access to function together.

- Create a failure-mode and effects analysis for each critical dependency.
- Use small blast-radius game days and Azure Chaos Studio where suitable.
- Measure detection, decision, failover, recovery, reconciliation, and customer communication times.
- Update architecture and runbooks from observed failures.

54.3 FinOps and Unit Economics

FinOps connects engineering, finance, procurement, and business ownership. Establish allocation through subscriptions, resource groups, tags, and cost centers. Forecast variable and commitment-based spend. Optimize idle resources, SKU, storage tiers, data transfer, reservations/savings plans where appropriate, licensing, and telemetry.

- Define unit cost such as cost per order, customer, environment, or data product.
- Create budgets and anomaly response before deployment.
- Rightsize from utilization and SLO, not from average CPU alone.
- Do not delete idle-looking resources without ownership, dependency, recovery, and retention checks.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Failover meets RTO but data is wrong	Replication lag, unprocessed messages, stale cache, split brain, or incomplete reconciliation	Stop further writes if necessary, quantify RPO breach, reconcile authoritative state, and improve failover gates.
Cost optimization causes latency	Capacity reduced below peak/SLO, cold start, throughput tier, or network path changed	Restore safe capacity, compare unit cost and SLO, then optimize with load tests and scaling limits.
Game day causes excessive impact	Blast radius, abort condition, communication, or dependency assumptions inadequate	Abort, restore, communicate, preserve evidence, and redesign experiment with narrower scope.
Budget alert is ignored	No owner, late threshold, inaccurate allocation, or alert fatigue	Assign accountable owner, tie to forecast and anomaly process, and escalate according to governance.

Hands-on Lab 54: Run a Well-Architected Review and Failure Game Day

Prerequisites

- A nonproduction representative workload from Parts A-D.
- Current architecture, cost data, SLO, dependency map, and recovery runbooks.

Implementation

1. Define business impact, SLO, RTO, RPO, peak load, data classification, and cost owner.
2. Review the workload against all five Well-Architected pillars and record risks and trade-offs.
3. Create a dependency map and failure-mode table with detection and recovery controls.
4. Choose one reversible failure such as dependency timeout, replica loss, or queue backlog.
5. Define hypothesis, blast radius, observation, abort threshold, roles, and communication.
6. Run the game day and measure detection, response, recovery, data correctness, and cost impact.
7. Create prioritized remediation backlog with owners and due dates; rerun the failed control.

Validation Evidence

- Review decisions link to requirements and evidence.
- Experiment remains inside approved blast radius.
- Recovery satisfies or quantifies deviation from objectives.
- Cost and reliability trade-offs are documented.

Cleanup

- Restore all test settings and remove chaos experiments.
- Archive evidence and close temporary elevated access.

Interview Questions and Model Answers

1. What is the purpose of Well-Architected review?

To identify risks and explicit trade-offs across reliability, security, cost, operations, and performance.

2. RTO versus RPO?

RTO is target restoration time; RPO is acceptable data loss expressed as time.

3. Why restore-test backups?

A successful backup job does not prove data can be restored, decrypted, connected, and validated within objectives.

4. What is FinOps unit economics?

Measuring cloud cost per business or technical unit so teams can connect architecture and usage to value.

Chapter Summary

- Apply Well-Architected pillars as trade-off disciplines.
- Prove resilience through measured recovery and game days.
- Define RTO/RPO/SLO and dependency behavior before choosing architecture.
- Manage cloud cost through ownership, forecasting, unit economics, and safe optimization.

Chapter 55 - Enterprise Azure Platform and Secure Delivery

Capstone

Integrating landing zones, Bicep, platform products, Azure DevOps, telemetry, Key Vault, Defender, Sentinel, network controls, resilience, and FinOps into one production operating model

Learning Objectives

- Compose an enterprise Azure platform architecture with clear platform and workload responsibilities.
- Implement secure source-to-production delivery with immutable artifacts and federated identities.
- Prove observability, security detection, governance, recovery, and financial controls.
- Produce portfolio-ready architecture decisions, runbooks, evidence, and executive risk summary.

55.1 Scenario: Regulated Digital Services Platform

Contoso Public Services must host multiple customer-facing applications in Azure. Workloads require separate subscriptions, private backend access, centralized egress, managed identities, strong audit, controlled deployment, data-residency rules, incident response, regional recovery, and transparent cost allocation. Development teams need self-service without becoming subscription owners or network administrators.

Capability	Reference Design
Enterprise foundation	Management groups, subscription vending, platform/application landing zones, policy initiatives.
Infrastructure delivery	Versioned Bicep modules, deployment stacks, what-if, protected environment checks.
Application delivery	Azure Repos/Artifacts/Pipelines, immutable promotion, WIF service connections, canary gates.
Secrets and keys	Key Vault/Managed HSM where required, managed identity, private endpoints, rotation.
Operations	Azure Monitor, Application Insights, OpenTelemetry, SLOs, workbooks, runbooks.
Security operations	Defender for Cloud, Sentinel, network controls, vulnerability and incident lifecycle.
Resilience and cost	RTO/RPO tests, game days, budgets, allocation, unit economics, optimization.

55.2 End-to-End Delivery and Control Flow

1. A product team requests an application landing zone with owner, classification, region, budget, and expiry metadata.
2. The vending pipeline places the subscription under the correct management group and applies policy, logging, security, network, and cost baselines.
3. Developers submit code and Bicep changes through protected pull requests using trusted templates.
4. CI runs tests and scans, creates SBOM/provenance, and publishes immutable artifacts.
5. A workload-identity service connection runs what-if and deploys to development.
6. Promotion to production requires external checks, security evidence, change approval, and SLO health.

7. Canary traffic is observed through traces, business SLIs, Defender/Sentinel signals, and cost telemetry.

8. Failure automatically stops progression; operators use audited rollback, replay, or recovery runbooks.

55.3 Capstone Evidence Pack

The final evidence pack should be understandable by an engineer, auditor, security analyst, finance partner, and executive. It must distinguish designed controls from tested controls and inherited platform controls from workload-owned controls.

- Context and requirements: users, data, regions, SLO, RTO, RPO, compliance, budget, and risk appetite.
- Architecture: management groups, subscriptions, identity, network, DNS, services, data flows, telemetry, and recovery.
- ADRs: platform choices, alternatives, trade-offs, assumptions, and review date.
- Delivery evidence: source, pull request, tests, scans, SBOM, artifact digest, what-if, approval, deployment, canary, rollback target.
- Operations: dashboards, alerts, SLOs, runbooks, on-call, patching, backup/restore, game-day results.
- Security: threat model, policy coverage, Defender findings, Sentinel detection, incident simulation, exceptions.
- FinOps: allocation, forecast, unit cost, anomaly response, and optimization backlog.

55.4 Acceptance Tests

Domain	Acceptance Test
Governance	Noncompliant region/public exposure is denied; approved exemption expires automatically.
Identity	No long-lived deployment secret; least-privilege WIF and managed identities are proven.
Delivery	Same immutable artifact moves through rings; failed health gate stops promotion.
Observability	One transaction traces edge-to-dependency and contributes correctly to SLO.
Security	Synthetic attack/control change creates owned Sentinel incident and safe playbook response.
Recovery	Restore or regional exercise meets measured objective or produces owned remediation.
Cost	Every major resource is allocated; budget/anomaly reaches accountable owner.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Platform deployment fails across subscriptions	Management-group policy, identity scope, provider registration, quota, or module version mismatch	Stop rollout, compare effective policy and deployment operations, fix at pilot scope, and rerun what-if.

Symptom	Likely Cause	Evidence and Corrective Action
Release passes CI but fails production check	Environment identity, protected-resource authorization, policy, network, SLO, or security signal	Do not bypass; inspect check evidence, correct dependency or approval, and redeploy same artifact.
Security alert lacks workload owner	Missing tags, CMDB mapping, subscription metadata, or entity enrichment	Improve vending metadata and Sentinel enrichment; assign interim incident owner and fix platform contract.
Regional exercise exposes hidden dependency	Global service, DNS, secret, identity, artifact, capacity, or external system not replicated	Document failure, update dependency map, choose resilience strategy, and rerun exercise.

Hands-on Lab 55: Deliver the Final Enterprise Azure Platform

Prerequisites

- Completed or representative resources from Volume 7 Parts A-D.
- Source repository, Azure DevOps project, sandbox subscriptions, and approved budget.
- Team roles for platform, application, security, operations, and finance.

Implementation

1. Write requirements and ADRs covering data, identity, network, SLO, RTO/RPO, security, governance, and cost.
2. Build a minimal landing-zone hierarchy and application subscription/resource-group baseline as code.
3. Publish versioned Bicep modules and create a protected Azure DevOps CI/CD path using workload identity federation.
4. Deploy a small application with private Key Vault access and complete monitoring.
5. Create SLOs, dashboards, alerts, Activity Log export, Defender review, and a Sentinel synthetic detection.
6. Enforce at least three policies and prove a time-bound exemption lifecycle.
7. Run canary failure, secret rotation, policy violation, security incident, and recovery/restore exercises.
8. Produce an evidence pack and a five-minute architecture-review presentation.

Validation Evidence

- All acceptance tests in section 55.4 have evidence.
- Every identity, resource, alert, policy, exception, and cost has an owner.
- No production-equivalent secret exists in source or pipeline storage.
- Known gaps are documented with risk, compensating control, owner, and deadline.

Cleanup

- Delete premium lab resources and temporary subscriptions according to policy.
- Remove elevated permissions and close exceptions.
- Retain approved source, evidence, diagrams, and lessons learned.

Interview Questions and Model Answers

1. What is the most important property of an enterprise cloud platform?

A repeatable operating model that makes secure, reliable, observable, and cost-accountable delivery the easiest path.

2. How do you prove a control works?

Trigger a realistic compliant and noncompliant scenario, observe enforcement and evidence, validate workload impact, and record owner and result.

3. What should block production promotion?

Missing immutable artifact evidence, failed tests/security/policy, unhealthy SLO or business signal, unauthorized identity, unresolved critical risk, or unavailable rollback.

4. How do platform and workload teams share responsibility?

The platform owns inherited foundations and paved roads; workload teams own application correctness, data use, release, SLO, and business operations within those guardrails.

Chapter Summary

- Integrate governance, delivery, operations, security, resilience, and FinOps as one system.
- Use federated identities and immutable artifacts from source to production.
- Prove controls through acceptance tests and failure exercises.
- Deliver evidence and ownership suitable for production, audit, and professional portfolio review.

Part E Command Reference

Commands are examples for a sandbox. Validate current CLI extension versions, API behavior, service availability, licensing, names, quotas, and organization policy before use. Replace placeholders and never paste secrets into shell history.

Area	Useful Inspection Commands
Bicep	<pre>az bicep build --file main.bicep az deployment sub what-if --location <region> --template-file main.bicep</pre>
Deployment stacks	<pre>az stack sub list --output table az stack sub show --name <stack></pre>
Azure DevOps	<pre>az devops configure --defaults organization=<url> project=<project> az pipelines list --output table</pre>
Azure Monitor	<pre>az monitor diagnostic-settings list --resource <id> az monitor metrics list --resource <id> --metric <name></pre>
Log Analytics	<pre>az monitor log-analytics workspace show -g <rg> -n <workspace> az monitor log-analytics query -w <workspace-id> --analytics-query <query></pre>
Key Vault	<pre>az keyvault show -g <rg> -n <vault> az keyvault secret list --vault-name <vault> --query "[].{name:name,enabled:attributes.enabled}"</pre>
Policy	<pre>az policy assignment list --scope <scope> --output table az policy state summarize --management-group <mg></pre>
Resource Graph	<pre>az graph query -q "Resources summarize count() by type"</pre>
Defender	<pre>az security pricing list --output table</pre>
Cleanup	<pre>az resource list -g <rg> --output table az group delete -n <rg> --yes --no-wait</pre>

Production-Readiness Checklist

- Bicep modules are versioned, tested, policy-aware, and reviewed through what-if before deployment.
- Deployment identities use workload identity federation or managed identity and have narrow environment-specific scope.
- CI agents are ephemeral or isolated; untrusted code cannot reach protected credentials or production networks.
- Artifacts are immutable and linked to tests, scans, SBOM, provenance, approvals, and deployment record.
- Production environments have independent checks, canary health gates, abort thresholds, and tested rollback.
- Azure Monitor collection, retention, sensitive-data filtering, alerts, runbooks, and telemetry-gap monitoring are owned.
- Critical journeys have reviewed SLIs/SLOs, error-budget policy, synthetic tests, and distributed trace correlation.

- Activity, Entra, policy, deployment, security, and network evidence is exported and retained appropriately.
- Key Vault/Managed HSM access, private DNS, recovery, rotation, expiry, and exposure response are tested.
- Defender recommendations and Sentinel incidents have owners, SLAs, exception rules, and cost controls.
- Landing-zone hierarchy, subscription vending, policy initiatives, remediation identities, and exemptions are code-managed.
- RTO/RPO, backup restore, regional recovery, game days, budgets, allocation, and unit costs are measured.

Cross-Service Troubleshooting Matrix

Symptom	Cross-Service Investigation
Pipeline authenticates but deployment is denied	Verify federated subject/audience, service connection, Azure role, policy/deny assignment, lock, scope, provider, and quota.
Deployment succeeds but application is unhealthy	Compare artifact digest, configuration, identity, Key Vault/DNS/network path, dependency status, probes, traces, and SLO.
No incident despite synthetic attack	Validate source event, connector/table, transform, analytics schedule, entity mapping, suppression, incident grouping, and automation.
Monitoring cost rises after release	Correlate diagnostic changes, log level, OTel cardinality/sampling, duplicate routing, new table retention, and failed retry loops.
Private dependency fails only in production	Resolve DNS from actual runtime, inspect routes/firewall/NSG, private endpoint state, service RBAC, and production policy.
Policy compliance differs from expectation	Check scope/exclusion, assignment version, mode/alias, evaluation delay, exemption, enforcement mode, and remediation status.
Regional recovery starts but cannot complete	Validate identity, DNS, key/secret availability, network, artifact registry, data RPO, messaging state, telemetry, capacity, and external dependencies.

Glossary

Term	Definition
ARM	Azure Resource Manager, the Azure control plane for resource deployment and management.
Bicep	A declarative Azure infrastructure language that compiles to ARM JSON.
Deployment stack	An ARM resource that manages a declared collection of resources and lifecycle/deny settings.
Template Spec	A versioned Azure resource used to publish an ARM template for controlled reuse.

Term	Definition
WIF	Workload identity federation, exchanging OIDC assertions for short-lived Azure credentials.
DCR	Data Collection Rule defining supported telemetry collection, transformation, and destinations.
KQL	Kusto Query Language used across Azure Monitor Logs and Microsoft Sentinel.
SLI	Service level indicator, a measured reliability or performance signal.
SLO	Service level objective, a target for an SLI over a period.
Error budget	The unreliability permitted by an SLO.
CSPM	Cloud security posture management for configuration and exposure risk.
SIEM	Security information and event management.
SOAR	Security orchestration, automation, and response.
Private Link	Azure private endpoint access to supported services through private IP addresses.
Landing zone	A standardized Azure environment foundation for operating workloads at scale.
Policy initiative	A collection of Azure Policy definitions assigned and managed together.
RTO	Recovery time objective.
RPO	Recovery point objective.
FinOps	A collaborative operating practice for cloud financial accountability and value.
ADR	Architecture decision record documenting context, options, decision, and trade-offs.

Official Microsoft Documentation Index

Reference	Microsoft Learn URL
Bicep documentation	https://learn.microsoft.com/azure/azure-resource-manager/bicep/
Deployment stacks	https://learn.microsoft.com/azure/azure-resource-manager/bicep/deployment-stacks
Azure Deployment Environments	https://learn.microsoft.com/azure/deployment-environments/
Azure DevOps documentation	https://learn.microsoft.com/azure/devops/

Reference	Microsoft Learn URL
Azure Pipelines security	https://learn.microsoft.com/azure/devops/pipelines/security/overview
Azure Monitor documentation	https://learn.microsoft.com/azure/azure-monitor/
Application Insights and OpenTelemetry	https://learn.microsoft.com/azure/azure-monitor/app/app-insights-overview
Azure Resource Graph	https://learn.microsoft.com/azure/governance/resource-graph/
Azure Update Manager	https://learn.microsoft.com/azure/update-manager/
Azure Key Vault	https://learn.microsoft.com/azure/key-vault/
Microsoft Defender for Cloud	https://learn.microsoft.com/azure/defender-for-cloud/
Microsoft Sentinel	https://learn.microsoft.com/azure/sentinel/
Azure Firewall	https://learn.microsoft.com/azure/firewall/
Azure Web Application Firewall	https://learn.microsoft.com/azure/web-application-firewall/
Azure landing zones	https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/
Azure Policy	https://learn.microsoft.com/azure/governance/policy/
Azure Well-Architected Framework	https://learn.microsoft.com/azure/well-architected/
Azure Cost Management and FinOps	https://learn.microsoft.com/azure/cost-management-billing/finops/

Azure capabilities, lifecycle status, portal experiences, limits, regions, pricing, and licensing change. Revalidate every production design against current Microsoft documentation, the Azure service health and pricing pages, and your organization's legal, security, compliance, and architecture standards.